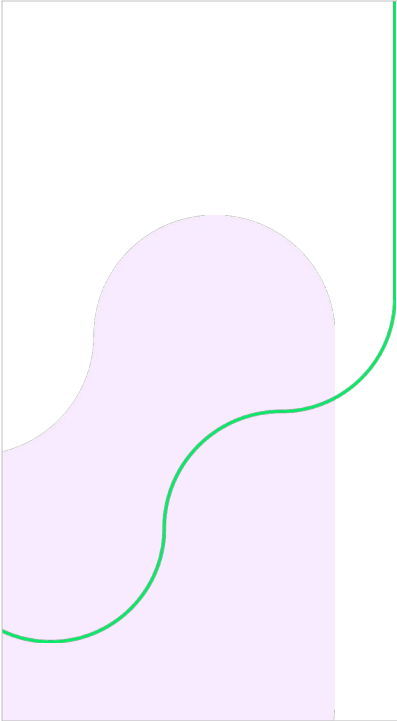




DS110

Atlas Search

Atlas Search and Vector Search



Topics we cover

Accessing Atlas Search

Atlas Search Indexes

Search operators

Combining operators

Autocomplete, Highlighting and
Synonyms

Performance and sizing



What do we mean by search?

"With a database query - you know exactly what you want.
When you use search you are open to suggestions." - unknown

```
db.movies.find({ title : "Raising Arizona"})
```

Title IS EXACTLY "Raising Arizona"

```
db.movies.find({ title: /Raising Arizona/})
```

Title CONTAINS the phrase "Raising Arizona."

```
db.movies.aggregate([{$search : { ..., text : { query : "Raising Arizona" , path: "title" } } }])
```

Title mentions some variations of "to Raise," "Arizona," or both

MongoDB is a database. As a database, it can safely perform storage and retrieval of records with guarantees of safety of interaction between multiple users and durability of writes and predictable interaction between them.

The \$search aggregation stage will also require an index to be specified for the Atlas search.

It's a primary data store and a system of record.

Atlas is MongoDB, hosted on public cloud, with day to day maintenance done for you by teams inside MongoDB - allowing you to focus on using the database, rather than running it.

So what is Atlas Search?

The "Read" in CRUD, which includes `find()` and `aggregate()` for MongoDB, identifies all documents exactly matching a specific criteria.

This match is boolean. A document matches - or does not - and the notion of matching is very exact.

Search, on the other hand, accepts that some things match better than others and allows us to order by how well they match.

This is most notable in filtering by text fields. Rather than an exact match, we can find where fields contain some of the query terms, or even similar query terms.



Scoring and Relevance

Search:

Title	Score
The Beginners Guide to Node.js	--
Introducing Javascript	29
Javascript for Dummies	25
Javascript the Definitive Guide	22
Introducing MongoDB	--

4

When we use Atlas Search, results are sorted not by a specific value in the data, like a normal database sort, but by those that match our query the best.

To do this, each result is assigned a "score." How exactly that score is assigned is something we have a lot of control over in Atlas Search.

This score is said to denote the "relevance" of a result, so by sorting and getting the highest score first, we can see the results our user will most want to see, the "most relevant."

Here, even though we don't have an exact match for "javascript beginner" in the database, we can find some relevant results.

We will see how to get these results using a Code Sandbox in a moment.

Who needs a text search capability?

The image shows a screenshot of an eBay search results page for the query "mountain bike gloves". The search bar at the top contains the text "mountain bike gloves". A callout box on the left, labeled "Mountain Bike Gloves", points to the search bar. Another callout box on the right, labeled "WFX Mountain Bike Bicycle Cycling Full Finger Gloves BMX MTB Riding Touchscreen", points to a specific product listing. The product listing shows a pair of black and yellow gloves for £6.99. The page also displays other search results, including a pair of gloves for £12.99 and a pair of gloves for £12.99 to £17.99. The page includes filters for price, size, and brand, and a sidebar with navigation links.

5

It's not hard to think of cases where a relevance-based text search is useful. Most of the time, when we type into a box on the internet to search for something, there is some degree of partial and relevance matching happening.

This could be something as obvious as an actual search engine - although large-scale internet search engines like Google determine relevance based on their data about the user as well as the content.

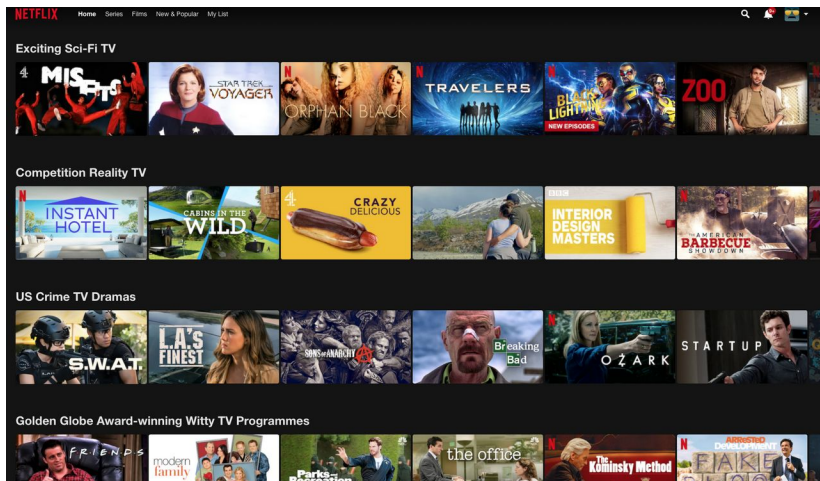
It could be an Ecommerce site - like Amazon or Ebay - where results default to sorting by "Best Match."

Or it could be more technical, like an online resource of academic papers or fiction, where high-quality text search is preferred.

One way to think about Search versus Query is this: Do I expect to interact with all the results, or just the first ones? And if the answer is "Just the first ones," what makes them first?

Not every data request is a search but in many cases, if a human is asking, then it is.

Who else needs text search capability?



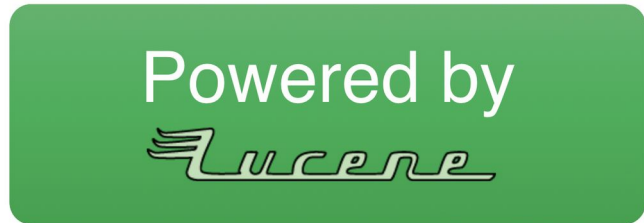
6

But there another common use case - recommendation engines, where rather than look at a user's specific query, you take the information you have about the user and then auto-generate a query to find the best matches.

This is less driven by something the user enters and more automatic based on the user's previous behaviours, although you can combine user input with things you know about them for personalised search results - or you can simply personalise their user experience automatically.



What is "Atlas" Search?



Atlas Search is an addition to the hosted MongoDB database allowing the creation of indexes to support Search using Apache Lucene, the world's leading text search technology.

Search allows you to make additional searchable indexes in a different format optimised for relevance searching.

Although this is something that can be built and configured yourself with a self-hosted MongoDB, it's complicated to do well. Users often have to install, configure, and pay for Elasticsearch or SOLR to do this.

With Atlas, it's as simple as specifying what data you want to search and the way you intend to search it. The indexes are then built and maintained for you in the background.

And as a developer, you can now perform relevance-ranked searching (and more) on behalf of your users.

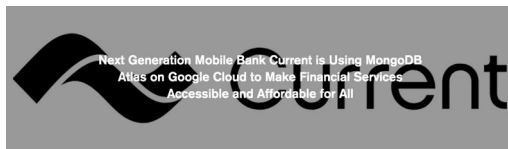
Best of all, it's all integrated, managed, and has no additional licensing costs or splitting of support between multiple organisations.



Who is using Atlas Search and why?

From banking to concert tickets to online fraud detection, Search is a powerful enhancement to Atlas.

humanitix



**Event ticketing
that's better for
Your Attendees**

100% of our profits fund education for disadvantaged kids.

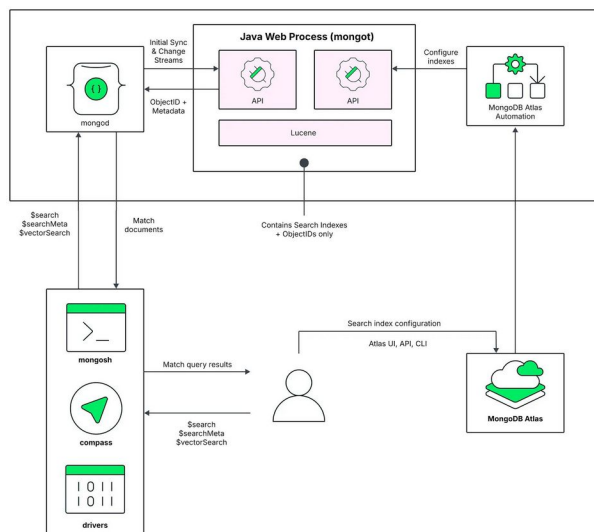
Search events

Host events

Current - a modern internet-first bank in the USA - provides Search over all account and transaction data. Data, which is held in Atlas anyway, uses Atlas Search to allow users to easily look through all of their financial information.

Humanitix event management and ticketing hosts its entire system on Atlas and provides Search to find the events customers are looking for.

Shared Node Architecture



9

When you enable Atlas Search on each server that is running a MongoDB database node, we add an additional process called MongoT.

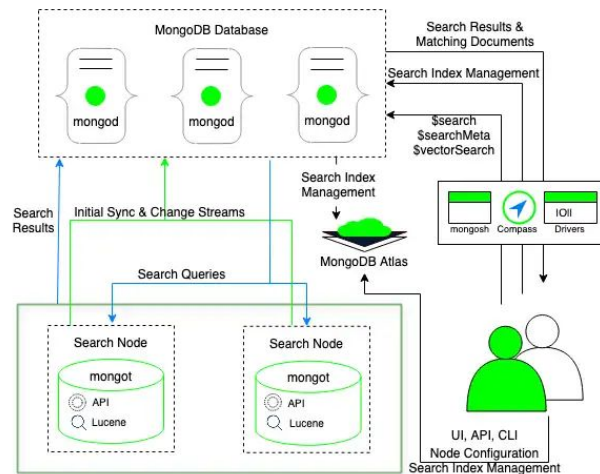
MongoT uses change streams to identify all relevant modification to the data in MongoDB. For each change event, if there is a relevant Lucene index defined, then the MongoT service updates the Lucene index.

All the configuration of this is managed via the Atlas UI or the Atlas DevOps APIs.

To search against the Lucene indexes rather than the normal WiredTiger indexes, we use special stage in Aggregation - specifying not a typical MongoDB query but MongoDB style representation of a Lucene query. The MongoDB database server (mongod) passes these search queries to MongoT, which then queries Lucene and passes back the `_id` values and any relevant metadata, like scoring.

MongoDB exposes as much native Lucene functionality as possible and keeps the MongoT layer thin. This makes it easy for us to expose new, state-of-the-art functionality in the underlying Lucene technology, unlike other companies that productize Lucene who are restricted by their own APIs and idioms on top.

Dedicated Search Node Architecture



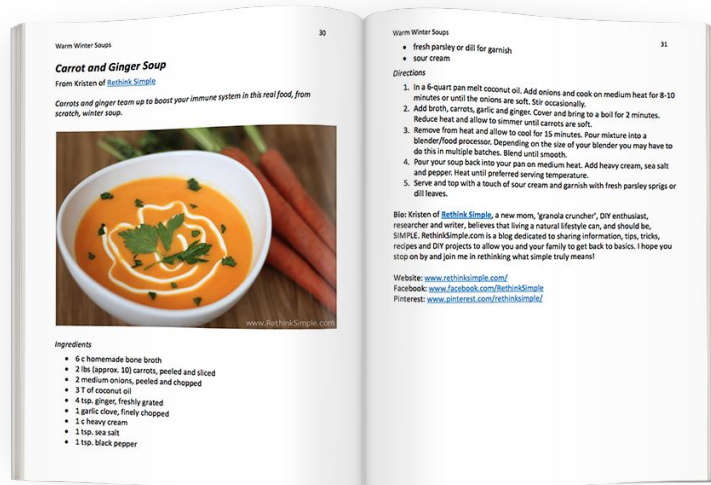
10

For production environments, we recommend a node architecture in which MongoDB processes and Atlas Search processes run on separate nodes.

This means your **mongot** processes run on nodes separate from your main database, so normal query operations and Atlas Search operations do not need to contend for resources.

You can scale your Search nodes vertically and horizontally, independently from your main database

Do we still need MongoDB or WiredTiger?



11

One interesting question you may ask is, "Do we still need MongoDB queries or WiredTiger storage? What for, when we have Atlas Search?" The answer is a definite yes.

Search is about data discovery, where you are looking for the best matches and reading them. Database queries are used where you want to be more definitive about getting all results or you want to order by specific fields, not relevance.

Lucene is an **indexing** technology, not a storage technology. My mother had a cookbook and at the back was an index by ingredients - so you could look for a recipe to use carrots, for example. Now you could, in theory, figure out from that index all the ingredients that went together in each recipe, but it wouldn't be helpful. You still need the recipe itself. Atlas search is that index. It's not enough on its own.

There is overlap and MongoDB, the company, will be seeking to use Lucene to optimise anywhere we can to provide the best user experience.

It's also very important to understand that the Lucene indexes are "eventually consistent." Unlike MongoDB, you cannot say that when you write something, everyone (including yourself) can immediately find it at that point via search. It may not become visible for some seconds,

MongoDB queries are therefore still used for all updates and data processing requirements.

Why isn't Atlas search just text search?

"fish restaurant"
AND "new york"
AND rating > 8

Powered by
Lucene

20,000 `_id` values for
fishy restaurants



Examines 20,000 Documents by `_id`



300 Results

12

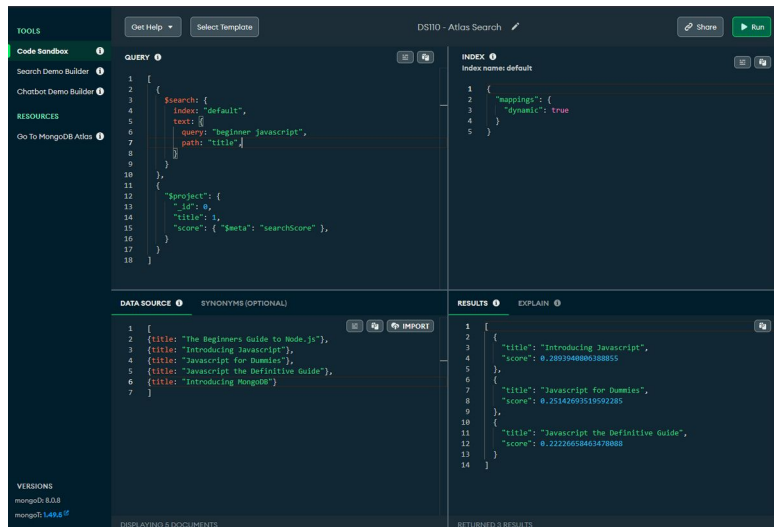
Atlas Search isn't just for text searches; it also supports exact matches, number/date range searches, geospatial searches, and various combinations of these. It can handle many of the same operations as standard MongoDB queries.

The reason for this extensive functionality within Atlas Search is efficiency. Performing all query, filtering, and ordering operations within a single technology using optimized indexes is much faster. For instance, combining a text search with a geospatial filter (e.g., "shops selling organic produce within 3 miles") or a date range (e.g., "emails about fishkeeping within a specific date range") is much quicker if Atlas Search handles it natively.

While you *can* split a query between Lucene (Atlas Search) and MongoDB, it's significantly slower because MongoDB would have to individually check each document returned by Lucene.



Atlas Search Playground



13

MongoDB provides a “Search Playground” which you can use to quickly experiment with Atlas Search settings. Here you can see how we generated the scores for the “Scoring and Relevance” slide.

You can try this out here: <https://search-playground.mongodb.com/tools/code-sandbox/snapshots/new>

The Search Playground allows you to share snapshots of your query, index and datasource to share with others. These snapshots are retained for up to 30 days.

You can use the following as the Query:

```
[{$search: {index: "default",text: {query: "beginner javascript",path: "title",}}}, {"$project": {"_id": 0,"title": 1,"score": { "$meta": "searchScore" },}}]
```

And the following as a Data Source:

```
[{title: "The Beginners Guide to Node.js"},{title: "Introducing Javascript"},{title: "Javascript for Dummies"},{title: "Javascript the Definitive Guide"},{title: "Introducing MongoDB"}]
```

Replace the Query with the following. How do the results change?

```
[{$search: {index: "default",text: {query: "beginner javascript",path: "title", fuzzy:{}}}}, {"$project": {"_id": 0,"title": 1,"score": { "$meta": "searchScore" },}}]
```

Accessing Atlas Search





Loading Sample Data in Atlas

Connect to an M0 Atlas cluster from a mongo shell
Add a small number of documents

If you don't have an M0 cluster or know how to connect with a terminal, your instructor can help you set one up

```
> use searchtraining
> db.example.drop()

//N.B. There are deliberate spelling errors below

> db.example.insertMany([
  { text: "This is a sentence and I can search for it using just one word" },
  { text: "This is a a different sentence, and I can search for it using one or two words" },
  { text: " Whether it is one word or two words could make a difference to the results" }
])

> db.example.find({})
```

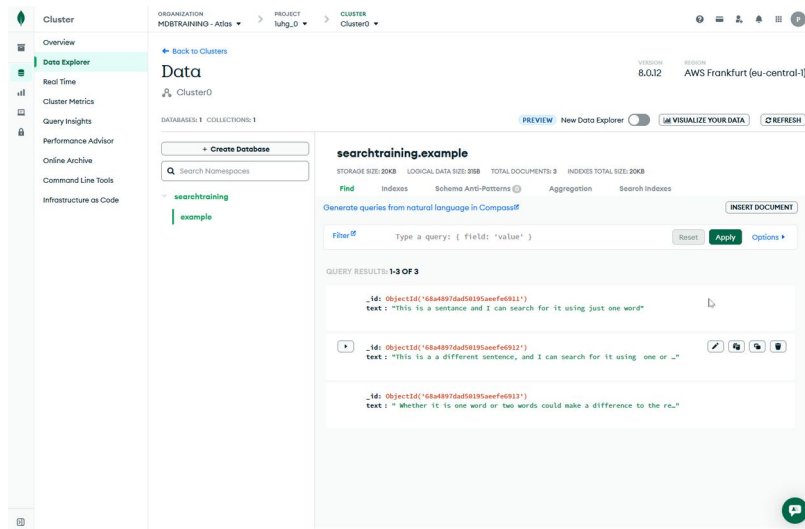
15

In order to best see the functionality of Atlas Search, we are going to add a tiny dataset. This makes it easier to understand our results in the context of every record in the collection.

Atlas Search works very quickly on any size of data but by using just a few documents, we can see what it does and does not return and how it scores them relatively.



View Documents in Atlas



Inside Atlas, use "Browse Collections" to take a look at the three documents you added, complete with deliberate spelling mistakes.

Creating a Default Search Index

Create a Search Index

1 Setup 2 Review

Start Your Index Configuration

Search Type
Which search type should I use?

Atlas Search
Full-text search for relevance-based app features.

Vector Search
For semantic search and AI applications.

Index Name and Data Source
Search indexes are specific to a database and collection.

Index Name
default

Database and Collection
Search for database or collection...

searchtraining
example

Configuration Method
Select how you would like to configure your index. You can also define indexes through the Atlas Admin API, Atlas CLI, MongoDB Shell, or supported Drivers.

Visual Editor
Define index definitions in a more guided experience.

JSON Editor
Edit the raw index definition with an extended JSON editor.

Cancel Next

Create a Search Index

1 Setup 2 Review

Review Search index "default" for searchtraining.example

VIEW JSON REFINE YOUR INDEX

You can edit these settings at any time to fine tune relevance and improve search performance. [Learn how to refine your index](#)

Index Configurations

Index Analyzer	Specifies how text is processed and tokenized when building the search index.	lucene.standard
Search Analyzer	Defines how search queries are processed and tokenized.	lucene.standard
Dynamic Mapping	Automatically maps supported data to field mapping types, including new fields and fields with polymorphic data.	On
Field Mappings	Configures specific fields with data types, analyzers, and input parameters. Field-level definitions override index-level definitions.	None
Stored Source Fields	Selects which fields from the collection are stored in the index for retrieval alongside search results. See Use Cases and Performance Considerations for details.	None
Synonyms Mappings	Respects defined word equivalents by accounting for similar terms (like "car" and "automobile") to improve search results.	None

Back Cancel Create Search Index

17

Now that we have inserted some documents and we can connect and see them in the GUI and via a shell, we need to tell Atlas we want to create a Search index for this data.

To do this, we click on "Search & Vector Search" and then "Create Search Index." As this is our first time, it takes us through a simple path.

We can use either a GUI editor (The Visual Editor) to configure our index or edit a JSON configuration file. For this first example, we will choose Visual Editor and click Next.

In the next screen, will keep the index name as "default." The index Atlas search uses this index if nothing is explicitly specified.

We need to unfold the **searchtraining** database and select the **example** collection.

We will accept all the default settings for now.

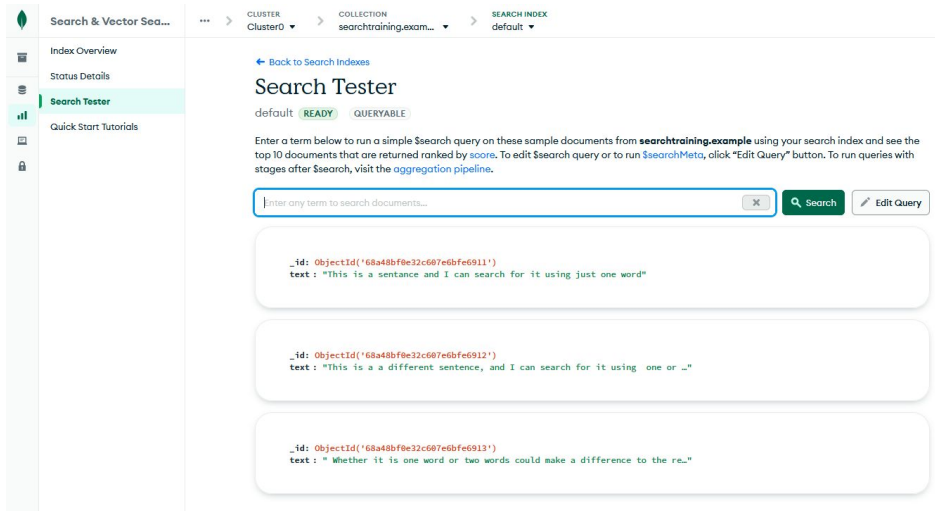
We are using the "Standard" text analyzer, so strings will be split into searchable values based on spaces and punctuations but will not know any specifics of the language.

We are using the "dynamic" mapping modes which means all number, text, and date fields will be indexed, including those in arrays or objects.

Click "Create Search Index" and then close the window that pops up. The index will take a few minutes to create.

We know it's done when it says "3 (100%) indexed of 3 total."

Performing a Simple Text Search



18

Once the index is built, then the "Query" button will be available and take us to the "Search Tester."

"Search Tester" is a very basic search application/GUI we can use to see how our search index would look or behave in an application.

Now we can type in a query, just as we would in any other search engine, and see matching results, scores, and highlighted terms.

Try searching for **"one"** - we can see documents that contain the word one in the context (where the word occurs) and also a score as to how good this was.

Try searching for **"one two"** - note we have a higher score for things with both one and two, but we still find the line with just one.

Try a search for **"sentence"** - note we do not find "sentence." Atlas Search can do this, but this GUI does not enable fuzzy searching. We cover that later.

Click "View Query Syntax" to see what it's doing. Note it's searching the index "default." It's a simple text query and the path (fields to search) is set to the wildcard: "*" which means every field in this index.

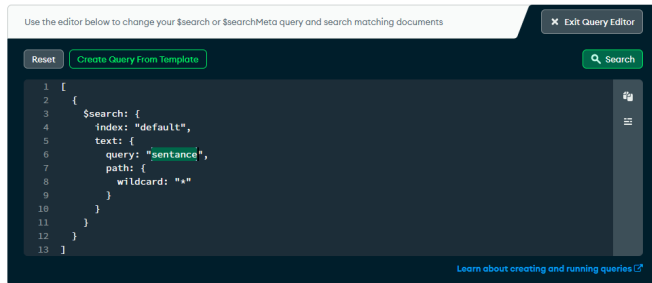


Searching from Code

Search Tester

default **READY** QUERYABLE

Use editor below to run \$search and \$searchMeta queries on documents from **searchtraining.example** using your search index. \$search queries return top 10 documents ranked by **score**. \$searchMeta returns a meta document. To run queries with stages after \$search or \$searchMeta, visit the [aggregation pipeline](#).



```
[{ $search : {  
  index: "default",  
  text : {  
    query : "sentence",  
    path : "text"  
  }  
}]
```

19

It's nice that the Atlas GUI gives us a specific search GUI, but that's just a basic, completed app. How can we perform this query in our code?

We can see that using the MongoDB shell, which is itself a REPL allowing us to run code and see the results. Using the connect details in your Atlas cluster, open up a MongoDB shell connected to your cluster. It will be a command like this one but with your unique cluster id and username.

```
mongosh "mongodb+srv://cluster1.kshqv.mongodb.net" --username myuser
```

Now, in the search tester in the GUI, with the intentionally misspelled word *sentence* typed in the box, click "View Query Syntax".

\$search is an **Aggregation** stage. It must be the first stage in the aggregation. If you are unfamiliar with aggregation, it's a set of transformative steps you can apply to documents before they are returned from the database.

We could add more stages like more matching, sorting, limiting the number of results, or rewriting the shape of the output by adding more pipeline stages.

We specify an **index**. Otherwise, it uses 'default.'

We are specifying a **text** operation (find values in text field).

The Search Tester performs a wildcard search, which is very performance risky in production environments. To be safe, we'll specify the exact field to search in, in this case "text"

In the the mongodb shell type:

```
use searchtraining  
query = <PASTE VALUE FROM GUI or SLIDE HERE>  
db.example.aggregate(query)
```

We can now see how easy it is to run a search from the shell or from any other driver.

We will be using the Mongo shell to demonstrate functionality in this class as we need to pass various options, and the Search Test UI has those hard coded for what it does.



Limiting Results

Atlas Search will by default return **all** relevant results.

Usually this is not what we want!

To reduce load on Atlas, always use `$limit`

```
> let query = [
  {
    '$search': { index: 'default', text: { query: 'sentence', path: 'text' } }
  },
  { '$limit': 20 }
]
```

```
Atlas [primary] searchtraining> db.example.aggregate(query)
[
  {
    _id: ObjectId("..."),
    text: 'This is a sentence and I can search for it using just one word',
    score: 0.46227604150772095
  }
]
```

20

Ordinary MongoDB queries return results through a cursor. MongoDB returns the results in batches, which means a single query cannot overload the server.

In Atlas Search, this does not happen. So we can easily send a query to Atlas Search which returns hundreds of thousands of results. This would tie up the Search instance for a long time.

As a best practice, always use the `$limit` aggregation pipeline stage in your Atlas Search queries. There's always a chance that our query might be malformed, or our database has more records than we thought.



Viewing the Result Relevance

We can see the score by using `$set` to add an additional output field

```
> let query = [ { '$search': { index: 'default', text:
{ query: 'sentence', path: 'text' } }
},
{ '$limit': 20 },
{ '$set': { score: { '$meta': 'searchScore' } } } ]

Atlas [primary] searchtraining> db.example.aggregate(query)
[
  {
    _id: ObjectId("..."),
    text: 'This is a sentence and I can search for it
using just one word',
    score: 0.46227604150772095
  }
]
```

21

The number in the search tester GUI, the score, shows how relevant our results are. In this case, relevance is judged on how often the value appears and also how many different fields it appears in.

We can also see on the list which fields it matched in. If a term is shown appearing in the same field multiple times, those will be different instances of the field as part of an array of objects.

To see this scoring information, we have to project a special metadata field called `searchScore`. We can do this with an additional pipeline stage where we add the score metadata to the output. We use `$set` to add a field to the aggregation output.

Now we can see a score for this result. If we had multiple matching results, they would be ordered by this score.



Exercise

You searched for the word "sentance" and output it with the score in the shell

Try to search for "one" and then "one two" and output those with the score in the shell



Creating Search Indexes in mongosh

Create Atlas Search (or Vector Search) indexes in mongosh

Provide the search index name and definition as in the GUI

View the index build progress using `$listSearchIndexes` or `getSearchIndexes()`

```
# Create single Search index
> db.movies.createSearchIndex(
  "searchIndex01",
  { mappings: { dynamic: true } }
)

# View status of search index builds
> db.movies.aggregate(
  [{ "$listSearchIndexes":
    {
      name: "searchIndex01"
    }
  }])

# View all search indexes in a collection
> db.movies.getSearchIndexes()

# Create multiple search indexes with one call
> db.runCommand( { createSearchIndexes: "movies",
  indexes: [
    { name: "searchIndex01", type: "search",
      definition: { mappings: { dynamic: true } } },
    ...
  ] } )
```

23

As of MongoDB v7.0 / v6.0.7 you can create Atlas Search and Vector Search indexes using the database command `createSearchIndexes`

Mongosh has a wrapper method `db.collection.createSearchIndex()` which allows you to create one Search index at a time.

By default, the method will create an Atlas Search Index with the name of 'default'. You can change this by supplying the index name.

You can also drop and update existing Search Indexes. See the documentation for examples: <https://www.mongodb.com/docs/manual/reference/method/db.collection.createSearchIndex/>

Atlas Search Indexes





WiredTiger (text) Indexes

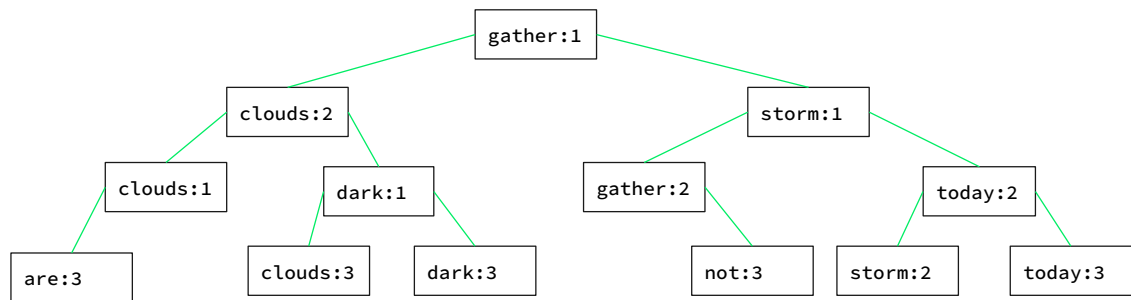
In WiredTiger, each index entry is a Key and a Single Document location

These are fast to update but slow to intersect

1 = { txt : "dark clouds gather storm" }

2 = { txt: "storm clouds gather today" }

3 = { txt: "today clouds are not dark clouds" }



25

We should start off by looking at how Search indexes and WiredTiger indexes differ to enable the different types of information retrieval.

This is being greatly simplified to explain the key points rather than going very deep into technical detail. In short, though, the data format enables the different type of use case.

WiredTiger indexes are BTree indexes, a type of binary tree optimised for on-disk storage and read and written as blocks. Every entry in these trees is a Key and a Value. The key is the value in the field of fields we are indexing. We can find a key in logarithmic time by starting at root of the tree and then looking for larger or smaller values until we find the value we need. At that point, we can read the matching document identifier and retrieve the document. WiredTiger uses something called Prefix Compression, so where a key is the same as or starts with the same as the previous one, we don't need to store the whole key, just the changes.



Lucene Indexes

Lucene uses an **inverted index** to get to a compact list of documents for a term that can be stored as multiple segments

Deletion and merging/compacting of these lists is done asynchronously

Slower to write but faster for multi-index queries, counting, and facets

1 = { txt : "dark clouds gather storm" }
2 = { txt: "storm clouds gather today" }
3 = { txt: "today clouds are not dark clouds" }

Inverted Index

id	Terms Dictionary	Postings List
1	are	1
2	clouds	1,2,3
3	dark	1,2
4		
5	gather	1,2
6		
7	not	3
8		
9		
10	storm	1,2
11	today	2,3

26

Lucene uses a "term dictionary" (essentially a hashmap) to point to a "postings list" (which contains a sorted list of document ids). This combination of a term dictionary and postings list is usually called an "inverted index."

Lucene postings files are very compact. They use variable encoded integers which can reduce disk IO and RAM needed to cache them. This allows faster traversal of the document list.

These "postings lists," being compact and sorted, require far fewer resources to use them for intersection (determining what documents occur in two or more lists) or to count the number of elements in them for facets and similar.

Except when using the legacy \$text indexes in MongoDB, a string field is a single index entry. By comparison, Lucene breaks a strings up into multiple terms and indexes them all independently.

This means a query for two words is performed in Lucene by looking up the index entry for each and then comparing two sorted lists of locations (Index intersection), making it quick to find and rank strings containing two or more words. WiredTiger does not normally support this sort of index intersection.

Lucene is slower to update its indexes than WiredTiger for three reasons:

- Each indexed field needs CPU to analyse and tokenize it and then there are multiple index updates, one for each word/token in the text.
- For each term, the postings list needs to have that document inserted at the right place in the list and stay sorted and compressed. It cannot be just added at the end.
- The posting list, despite being broken up into smaller "segments," still needs to write a lot more data for an index change than WiredTiger does.

To optimize disk and CPU usage during updates, Lucene performed some housekeeping tasks asynchronously, like merging document list segments and deleting older data.



How Atlas Search indexes BSON data types

BSON Type	Included in Dynamic Index?	Atlas Search Field Type
Double	yes	number
32-bit integer	yes	number
64-bit integer	yes	number
String	yes	string
Date	yes	date
Object	yes	document
Array	yes	type of elements
ObjectId	no	objectId
Boolean	yes	boolean

27

We've talked mostly about full text indexes, and that is the normal thing people think about with Atlas search.

The way MongoDB has integrated with the Lucene search technology, we can choose how to index individual fields, should they be treated as text for "contains" searches or be treated as dates, or as numbers and should numbers be integer or floating point.

Atlas search has a number of different field index types—Boolean indexes, Date indexes, geospatial indexes, Strings—including full text indexes or indexing the whole field as one value.

When we created our first index, we chose dynamic mapping. In this case, Atlas Search makes an index for all database fields of the following types: Double, Int32, Int64, String, Date, Object, and Array. In objects, each individual property is indexed like a MongoDB wildcard index. For arrays, each array member is indexed. Unlike a compound index in MongoDB, each field is indexed separately and can be searched separately.

For indexing arrays, Atlas Search only requires the data type of the array elements. You don't have to specify that the data is contained in an array in the index definition.

When a document type (object) is dynamically indexed, then all its children are also dynamically indexed.

If we want to index ObjectIDs, then we need to create an explicit mapping of each field in our index rather than a dynamic mapping.

Atlas Search will not index any of the other MongoDB types such as Binary Data, Nulls, or Decimal128.



Dynamic vs Explicit Field Mappings

Dynamic Field mappings index
all fields

Usually this is not what we want!

Explicit field mappings let us
choose which fields to index

Or dynamically index everything
below a certain key

```
> use searchtraining
> db.example.drop()
> let newdocs = [
  {
    activity: 'Climbing mountains',
    hours: 12,
    people: [ 'john rider', 'fiona wilson' ],
    note: 'watching eagles up Ben Nevis',
    start_location: { type: 'Point', coordinates: [ -8.61, 41.14 ] }
  },
  {
    activity: 'riding bikes',
    hours: 5,
    people: [ 'john rider', 'richard', 'fiona wilson' ],
    start_location: { type: 'Point', coordinates: [ -3.256, 40.364 ] }
  },
  {
    activity: 'watching television',
    hours: 16,
    people: [ 'John jones', 'carol smilie' ],
    note: 'watched netflix',
    start_location: { type: 'Point', coordinates: [ -8.61, 41.14 ] }
  },
  {
    activity: 'watching youtube',
    hours: 20,
    people: [ 'Fiona wilson' ],
    start_location: { type: 'Point', coordinates: [ -5.67, 10.19 ] }
  }
]
> db.example.insertMany(newdocs)
```

28

An index can use dynamic field mappings. Dynamic field mapping is a way to index a document where every field in the document with an appropriate data type is indexed, including arrays and nested documents. This can result in a large index, possibly indexing fields we do not need to search.

You might want more precise control over which fields are searchable and how they are searched. In this case, you should use explicit field mapping. You can still choose to dynamically map other parts of your data, including arrays and objects (and arrays of objects).

To see this, we will replace our documents with something with a few more data types and fields.

In your shell, do the following. Dropping the collection will also drop any existing Search indexes.

```
use searchtraining
db.example.drop() // This will drop the Atlas search index we created too
newdocs = [
  {_id:1, activity: "Climbing mountains", hours: 12 , people: ["john rider","fiona wilson"], note: "watching eagles up Ben Nevis", start_location:{type:"Point", coordinates:[-8.61,41.14]}},
  {_id:2, activity: "riding bikes", hours: 5, people : ["john rider","richard","fiona wilson"], start_location:{type:"Point", coordinates:[-3.256,40.364]}},
  {_id:3, activity: "watching television", hours: 16, people : ["John jones","carol smilie"],note: "watched netflix", start_location:{type:"Point", coordinates:[-8.61,41.14]}},
  {_id:4, activity: "watching YouTube", hours: 20, people: ["Fiona wilson"], start_location:{type:"Point", coordinates:[-5.67,10.19]}}]
db.example.insertMany(newdocs)
```

Creating an index with Field Mappings

```
{  "mappings": {
    "dynamic": false,
    "fields": {
      "activity": {
        "type": "string",
        "analyzer": "lucene.english"
      },
      "hours": {
        "type": "number"
      },
      "people": {
        "analyzer": "lucene.keyword",
        "type": "string"
      }
    }
  }
}
```

Create a Search Index



Review search index "default" for searchtraining.example

[View Atlas Search Docs](#)

```
1 {
2   "mappings": {
3     "dynamic": false,
4     "fields": {
5       "activity": {
6         "type": "string",
7         "analyzer": "lucene.english"
8       },
9       "hours": {
10        "type": "number"
11      },
12       "people": {
13         "analyzer": "lucene.keyword",
14         "type": "string"
15       }
16     }
17   }
18 }
```

Back

Cancel

Create Search Index

29

Now we want to create an explicit index. We want to be able to search the activity, the hours doing it, and the participants.

We do not want the notes to be searchable. Whilst we will talk more about this later, the index size is dependent on the number of fields in the index. So, by not including a field in the index, we make it smaller. Smaller indexes need less RAM in the server and so cost us less to host.

We may also not want matches based on some fields, or we may want them in a different index altogether. By using an explicit mapping, we can do this.

We will use the JSON editor. We could configure this via the Visual editor, but it takes longer and there is more chance of making a mistake. In production, we would probably store our index definitions in our source code control as JSON anyway, and make changes via the API.

Create an index called 'default' and select searchtraining.example:

- Set *dynamic* to *false*, as we only want to index the listed fields and only the types we specify.
- Set our analyzer to english for the activity, but not for the array of names.
- For the people's names, use the keyword analyzer. This will store each string as-is.
- Set hours as a number, and we do want it indexed. There are many other options we can use to refine this in the future.

Unlike compound indexes, the order of fields in the Atlas Search index definition is not important. Fields can be defined in any order as each is a separate index structure that can then be intersected.



Analyzers and Tokens

Text to index:

"Your time is limited, so don't waste it living someone else's life."



lucene.english analyzer

your	time	limit	wast	liv	someon	else	life
------	------	-------	------	-----	--------	------	------

Search text with
high matching score:

"You're wasting your time by trying to live the life of someone else."

One of the largest differences between Atlas Search and MongoDB is the options for handling text fields, values where the BSON data type is string. In Atlas Search, you can define how a string is broken down into individual indexed parts (known as tokens). So you need only match parts of the string—not the whole string. The more you match, the higher the score. Tokens might typically be words or prefixes of words.

We can also perform some normalisation on this—for example, converting all the tokens to lowercase, converting accented characters to their unaccented versions, and stemming, which means removing word endings so words such as run, running, and runner all end up as the same value.

The component that performs the tokenisation is the "Analyzer". For our index, we chose the "lucene.english" analyzer for the "activity" field. This allows it to apply language-specific rules such as stemming (storing shorter versions of the word in the index with the tense suffix removed).

In this above example, we can see the original text and how it was indexed into tokens by the lucene.english analyzer. The search text below matches every single token except for "limit", so it would rank as highly similar to the original text.

Note that Atlas Search is not concerned with context or meaning, just words. We can combine Atlas Search with Atlas Vector Search for context-based matching. More of that is in the Hybrid Search module of this course.



Analyzers and Tokens

People uses a keyword analyzer, so it needs exact matches

Activity uses an English analyzer and is able to find sub parts of the string or similar strings

```
> use searchtraining
> query1 = [ { $search: { index: 'default', text: { query: "john", path: "people" } } }, { $set: { score: { $meta: "searchScore" } } } ]
> db.example.aggregate(query1).pretty()

> query2 = [ { $search: { index: 'default', text: { query: "John Rider", path: "people" } } }, { $set: { score: { $meta: "searchScore" } } } ]
> db.example.aggregate(query2).pretty()

> query3 = [ { $search: { index: 'default', text: { query: "john rider", path: "people" } } }, { $set: { score: { $meta: "searchScore" } } } ]
> db.example.aggregate(query3).pretty()

> query4 = [ { $search: { index: 'default', text: { query: "ride", path: "activity" } } }, { $set: { score: { $meta: "searchScore" } } } ]
> db.example.aggregate(query4).pretty()
```

31

Let us run some queries to see our new, more refined index in action:

```
use searchtraining
query = [ { $search: { index: 'default', text: { query: "john", path: "people" } } }, { $set: { score: { $meta: "searchScore" } } } ]
db.example.aggregate(query).pretty()
```

You might be surprised to find we don't find anything. We used the 'keyword' analyzer for the people field, so that indexes the whole string without breaking it up. We also have to match the case exactly.

```
query = [ { $search: { index: 'default', text: { query: "john rider", path: "people" } } }, { $set: { score: { $meta: "searchScore" } } } ]
db.example.aggregate(query).pretty()
```

```
query = [ { $search: { index: 'default', text: { query: "John Rider", path: "people" } } }, { $set: { score: { $meta: "searchScore" } } } ]
db.example.aggregate(query).pretty()
```

Compare this to the query below—where a query for "Ride" finds the word "riding"—because this was indexed as English and it can tell that "riding" is a variation of "ride."

```
query = [ { $search: { index: 'default', text: { query: "ride", path: "activity" } } }, { $set: { score: { $meta: "searchScore" } } } ]
db.example.aggregate(query).pretty()
```



Built-in Analyzers

Original Text:

José is much taller, his email is j.parillo@gmail.es.

Analyzer	Tokens after Analysis
Standard	josé much taller his email j.parillo gmail.es
Simple	josé is much taller his email is j parillo gmail es
Whitespace	José is much taller, his email is j.parillo@gmail.es
Keyword	"José is much taller, his email is j.parillo@gmail.es." (One value!)
English	josé much tall hi email j.parillo@gmail.es
Spanish	josé is much taller his email is j.paril gmail.

32

There are five built-in analyzers (with variations of the language one for what language is used):

- The **Standard** Analyzer divides on word boundaries, converts to lowercase, and removes punctuation. It is usable for most languages and recognizes email address, acronyms, alphanumerics, and Chinese-Japanese-Korean (CJK) characters. It also has standard English stopwords.
- The **Simple** Analyzer slits text on whitespace or punctuation and converts everything to lowercase.
- The **Whitespace** analyzer splits just on whitespace and does not modify case.
- The **Keyword** Analyzer treats the entire text as a **single** term.
- Language-specific Analyzers are available for 42 languages and they are tailored to tokenizing in a way appropriate to the specific language.
 - English will not remove diacritics like the accent in José but will stem "taller" to "tall" and remove English stop-words like "is."
 - In Spanish, **his** is not stemmed. Spanish won't remove English stop-words like **is** either.



Custom Analyzers

You can create a custom analyzer by configuring:

Character filters - Removing accents, standardizing characters

Tokenizing of strings - Understanding language context

Token filtering/modification - Synonyms, removing short words, etc.

33

We can also specify custom Analyzers if we want, to convert our text fields to normalized tokens in a specific way.

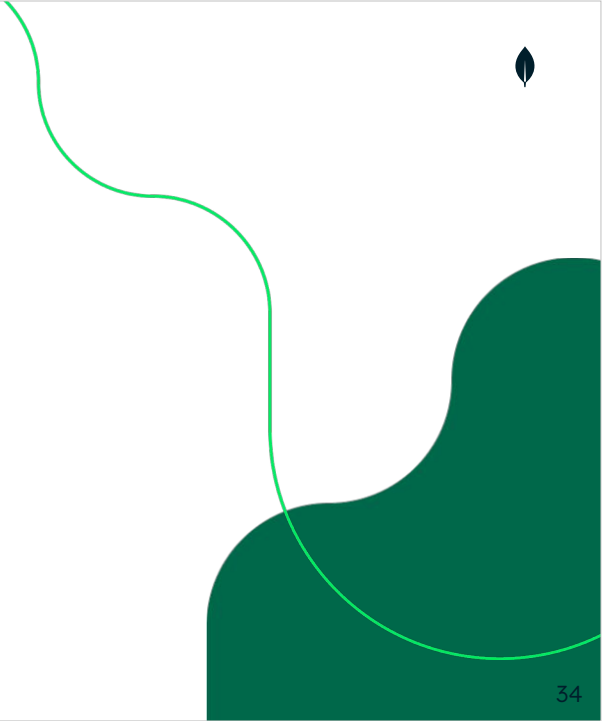
When we define a custom analyzer, we can specify:

- Character filters: Remove/replace/transform individual characters (remove HTML, normalise, map),
- Tokenizer: Specify how to break into tokens—standard ones, regex based ones, ngram and edgogram (see later).
- Token filters stemming/replacements: Soundalike, ngrams, length, regex, lowercase, stopword.

We are not going to go into depth today on how to do this, but you have a lot of flexibility to specify how your incoming strings should be split up for searching by parts.

Custom Analyzers are not available on M0 shared-tier clusters.

Search Operators



34

We have shown the text search operator. That's the one we usually think of when we think about the idea of search. We can do much more with Atlas Search to compute relevance, range searches, exact searches, geospatial searches, and combinations of these. We are going to go over the key search operators now.



The text Operator

Atlas Search:

```
{ $search : { text : { query : "watching television", path: "activity" } } }
```

MongoDB Regex - Not Indexed!:

```
{ activity : /watching|television/ }
```

```
query = [ { $search: { text: { query: "watching television", path: "activity" } } } ]
db.example.aggregate(query).pretty()
```

```
//This finds bikes
```

```
query = [ { $search: { text: { query: "hike", path: "activity" , fuzzy : { maxEdits:1} } } } ]
db.example.aggregate(query).pretty()
```

```
//Insert this new document
```

```
db.example.insertOne({activity:"tracking elephant",hours:3,people:["Alan Quartermain"], start_location:{type:"Point", coordinates:[-8.61,41.14]}})
```

```
//This query will not find the previous document! Why not?
```

```
query = [ { $search: { text: { query: "Elephant", path: "activity" , fuzzy : { maxEdits:2} } } } ]
db.example.aggregate(query).pretty()
```

35

The operator we have used until now is the **text** operator. This is what we have seen in the Search Test GUI, and we saw it used at the command line. It is used to find matching strings, where matching is scored based on how many of the tokens in the string are in the query. The closer the match, the higher the score.

The closest alternative to this in traditional MongoDB is to use a regular expression (\$regex) or the MongoDB \$text search, but there is really no good alternative natively in MongoDB. This is the core reason to use Atlas Search.

When we perform a search, we either specify an alternative index we want to use, or we will use the index called **default**. We then specify the operator we want to use to compute the scores. If this is the text operator, then we have some mandatory and potentially optional parameters.

query - A simple string we want the best possible match to; how many tokens/terms in the search string are in the index. This is mandatory.

path - What field or fields we want to compare our string to. It can be specified as a wildcard to match multiple fields. This, too, is mandatory.

The Query String is analyzed to get terms. Each document is then scored based on the presence of each of those terms in the index, the scarcity of the terms, and the length of the field they were in.

Optional **"fuzzy"** match options as an object—these allow us to specify how many character differences we will accept in token. For example, grey and gray are one difference apart. We can also specify how much must match at the start, so with a prefix of 2, then grey == gray but pray != gray. If we add a fuzzy option, we can find "zoo" when searching for "soo." We specify how many characters can be modified, added, or removed: either 1 or 2. In the last example, the user has made a mistake and searched for "Elephant," and we allowed a "Fuzzy match with up to two changes." Nevertheless, this query does not match any documents. Why not? (Hint: the analyzer we chose for the 'activity' path will stem longer words.)

MongoDB Regex - we may also create a text index and search for { \$text : { \$search : "watching television" } }



The range operator

```
search = [ { $search: { range: { path : "hours" , gt: 10} } } ]
db.example.aggregate(search).pretty()

query = { hours : { $gt: 10}}
db.example.find(query)

search = [ { $search: { range: { path : "hours" , gt: 10, lt: 20} } } ]
db.example.aggregate(search).pretty()

query = { hours : { $gt: 10, $lt: 20 }}
db.example.find(query)

//Exact Match on a Number needs a double range but you can also use the "equals" operator
search = [ { $search: { range: { path : "hours" , gte: 12, lte: 12} } } ]
db.example.aggregate(search).pretty()

query = { hours : 12}
db.example.find(query)
```

36

The range operator is used for numeric or date fields to find things within a given range. It can be used for greater than, less than, or both.

You also use the range operator for an exact match for date or numeric types. The "equals" operator can also be used to exactly match a number or date.

You specify the **query** and **path** in the same way as the other operators.



The near operator

```
search = [ { $search: { near: { path : "hours" ,  
                                origin: 12,  
                                pivot :2} } },  
            { $set: { score : { $meta: "searchScore" }}}}  
db.example.aggregate(search).pretty()
```

Near can be used for numbers, dates, or geolocations - The closer the 'Origin,' the higher the score

Pivot determines how the score drops as you move away from the origin

Value	12	10 or 14	8 or 16	6 or 18	4 or 20
Score	$(2/2) = 1.0$	$(2/4) = 0.5$	$(2/6) = 0.33$	$(2/8) = 0.25$	$(2/10) = 0.2$

37

The **near** operator works with numeric, date, and geographic fields.

It allows you to assign a score relative to how close the value in the document is to the value given in the query, unlike range, which is simply a boolean match. The near varies its score based on how close the match is.

You specify the **origin**—the value you want it to be near—and a '**pivot**,' which determines how quickly the score should drop off.

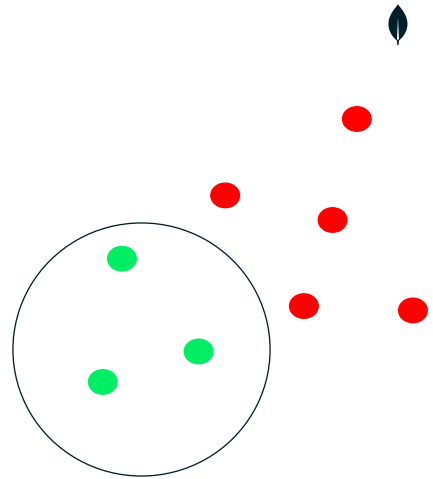
An exact match is equal to a score of 1.0.

At '**pivot**' distance away that drops to 0.5 according to the formula $\text{score} = \text{pivot}/(\text{pivot} + \text{distance})$ where $\text{distance} = \text{abs}(\text{origin} - \text{value})$.

geoWithin

```
search = [{
  "$search": {
    "geoWithin": {
      "path": "start_location",
      "circle": {
        "center": {
          "type": "Point",
          "coordinates": [-8.61, 41.1]
        },
        "radius": 16000
      }
    }
  }
}]
```

```
db.example.aggregate(search).pretty()
```



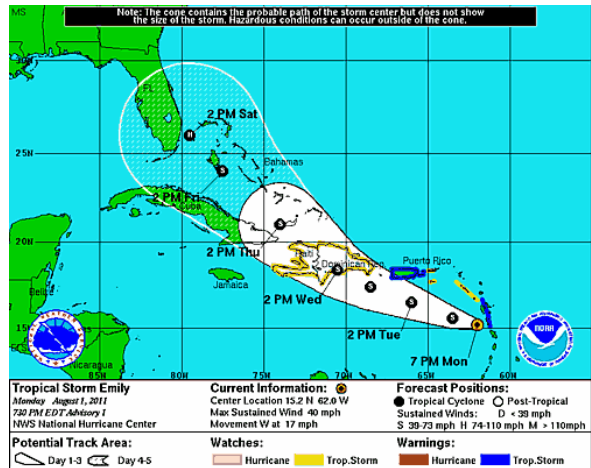
You can include scoring based on geographic relative position in your query using **geoWithin** and **geoShape**.

geoWithin is used where the documents contain geoJSON points (the most common option in geolocated data).

You specify a shape using GeoJSON and query where points lie within that. The shape can be a circle, a box, or an arbitrary geometry.

geoShape

```
search = [ { $search: {  
  geoShape: {  
    path : "countryshape" ,  
    relation: "intersects",  
    geometry: {  
      ... Long set of info ...  
    }  
  }  
}  
}]  
  
db.example.aggregate(search).pretty()
```



39

Note: This is just an example, there is not countryshape in the example collection
Unlike geoWithin, geoShape is used where the documents contain geoJSON objects and the indexShapes flag was added when creating the index definition.

Using geoShapes, you can then query for documents where the shape specified in the query contains, is contained within, intersects, or does not intersect the ones in the documents.

In this example, imagine we have a geoJSON geometry describing the predicted path of the hurricane. This would be a polygon.

We can then ask for a list of those countries the are completely covered by it, those that are partially covered by it, and those it does not touch at all.



The wildcard operator

```
use searchtraining

search = [ { $search: { wildcard: { path : "people" ,query: "joh*" } } } ]
db.example.aggregate(search).pretty()

//This is an error - activity has analyzed not raw values in
search = [ { $search: { wildcard: { path : "activity" ,query: "wat*" } } } ]
db.example.aggregate(search).pretty()

//This works - as we are stating we know it's analyzed
search = [{ $search: { wildcard: { path: "activity" ,query: "wat*" , allowAnalyzedField
: true }}}]
db.example.aggregate(search).pretty()
```

40

The **wildcard** operator is used to match where you do not know the exact value and want to add wildcards to the query.

? matches one character and * matches any number of characters.

With the **wildcard** operator, the query is not analyzed/tokenized, so this is usually used to match fields using the keyword analyzer.

If you want it to be able to query against a field that has been analyzed and split up (so you are comparing to individual terms), then you need to specify that explicitly using the flag **allowAnalyzedField**. For example, we can look in our activity field, which has stemmed English words for “watching.”



The equals operator

For DocumentID, numbers, dates, and boolean values, we can do an exact match using equals

This returns a score of 1 if true and 0 if false

```
MongoDB > use searchtraining

MongoDB > id = db.example.findOne()._id
ObjectId("634424519e25976c14ac3a96")

MongoDB > search = [{ $search: { equals: { path : "_id"
,value:id}}}]

MongoDB > db.example.aggregate(search).pretty()
//Nothing as we haven't indexed _id!
```

41

Atlas Search includes a simple equality operator, with a default score of one for a match. You can use the equals operator to query booleans, objectIds, numbers, and dates, however, they need to be explicitly included in the index you are querying against.

In our example here, we have not indexed `_id`, so this fails to find anything, but it demonstrates what would happen if we did have an index. Default indexing does not include boolean or objectId values.



Other operators

There are several more of these basic matching operators available:

exists – exists in a document

regex – contains a value matching a regular expression

phrase – contains an ordered sequence of terms (allowing for different spacing or even words between)

Lucene has many query operators. Atlas makes most of them available, from doing regex matches to requesting terms be near each other and even scoring based on whereabouts in the text they appear at the start or end.

The MongoDB Atlas Search documentation is a good source of reference on what the latest set of operators are.

Regular expression should only be used where you cannot use wildcard as an operator as it is less efficient.



Modifying the score for operators

```
showscore = { $set: { score: { $meta: "searchScore" } } }

//range is a boolean query, so scores 1 or 0
search = [ { $search: { range: { path : "hours" , gt: 10} } }, showscore]
db.example.aggregate(search).pretty()

//Boost lets us multiply that, so these score 2, but something with a score of 0.2 would be 0.4
search = [ { $search: { range: { path : "hours" , gt: 10, score: { boost: { value: 2 } } } } }, showscore]
db.example.aggregate(search).pretty()

//By boosting with a path, we can multiply by the field value
search = [ { $search: { range: { path : "hours" , gt: 10, score: { boost: { path: "hours" } } } } }, showscore]
db.example.aggregate(search).pretty()

//Or we can use a constant value. This makes sense for non-boolean operators
search = [ { $search: { range: { path: "hours", gt: 10, score: { constant: { value: 3 } } } } }, showscore]
db.example.aggregate(search).pretty()
```

43

All operators have a default way of scoring. For some, like range and equals, it is 0 or 1 depending on whether it does or doesn't match.

For others, like text and near, it is variable. We can adjust the score or any of our operators by adding boost and/or constant to the query.

boost multiplies the score by a fixed value or by the value of another field. Boost by itself will have no effect on the order of the returned results!

constant simply replaces the score with a constant. It is non-zero.

We can modify the score based on the field that matched or even modify the score to include the value of a field using the "**path**" modifier.

If you require even more complex and tailored scoring, you can explicitly define a function to compute it based on field values and any relevance scoring.

This is most useful when we are combining multiple operators together to adjust the relative weights of them. We will see how to combine them in the next section.

Combining Operators





Combining multiple criteria

Using the **compound** operator, we can combine search operations with boolean logic:

must – All clauses have to match, equivalent to AND

mustNot – None of the clauses match, equivalent to AND NOT

should – N terms must match from the set, where N defaults to 0

filter – The same as *must*, but does not affect the score

45

When using **should**, if you are only using a **should** clause, although none of the terms need to match, without a match the score is 0 and the document is not returned.

So **should** alone functions like a logical OR.



The compound operator using must

```
//One Clause
search = [ { $search: { compound : { must : [
  { range: { path : "hours" , gt: 10 } }
]}}} ]

db.example.aggregate(search).pretty()
//3 results

//Two Clauses AND'ed with must
search = [ { $search: { compound : { must : [
  { range: { path : "hours" , gt: 10 } } ,
  { text : { path: "activity", query: "ride" }}
]}}} ]

db.example.aggregate(search).pretty()

//0 results - the ride was only 6 hours
```

46

The compound operator lets us group multiple conditions together, with each contributing to the scoring. There are different types of compounding which introduce boolean logic to whether a document is returned.

Here we are looking at **must**, which works like AND logically: All listed clauses must be true. In the example, we show the difference between one and two clauses and that adding a further restriction reduced the number of documents returned.



The compound operator using mustNot

```
//One Clause
search = [ { $search: { compound : { mustNot : [
  { range: { path : "hours" , gt: 10 } }
]}}} ]

db.example.aggregate(search).pretty()
//2 result

//Two Clauses AND'ed with mustNot
search = [ { $search: { compound : { mustNot : [
{ range: { path : "hours" , gt: 10 } } ,
{ text : { path: "activity", query: "ride" } }
]}}} ]

db.example.aggregate(search).pretty()

//Not (hours > 10) AND Not (Activity contains "ride") - Only 1 result
//1 result: {activity: 'tracking Elephant', hours : 3}
```

This returns exactly the inverse of the documents on the previous slides.



Combining with compound using should

```
//One Clause
search = [ { $search: { compound : { should : [
  { range: { path : "hours" , gt: 10 } }
]}}}

db.example.aggregate(search).pretty()
//3 results as result need a score>0 to be returned

//Two Clauses OR'ed
search = [ { $search: { compound : { should : [
  { range: { path : "hours" , gt: 10 } } ,
  { text : { path: "activity", query: "ride" } }
]}}}

db.example.aggregate(search).pretty()
// 4 results
```

48

Unlike the **must** operator, the **should** operator on its own functions like a logical OR. One or more of the clauses must have a non-zero score to evaluate to any score.

This is equivalent to MongoDB's \$or operator.

When used with another operator, then, should adds to the score but none of the terms are required, as the other operator will create a score.

You can also set the number of terms required to match in a should clause using "minimumShouldMatch" to say that N of these terms are required.



Combining with filter

```
showscore = { $set: { score: { $meta: "searchScore" } } }

//Two Clauses AND'ed with must

search = [ { $search: { compound : { must : [
{ range: { path : "hours" , lt: 10 } } , { text : { path: "activity", query: "ride" } }
] } } }, showscore ]

db.example.aggregate(search).pretty()

//Two Clauses AND'ed with filter SCORE is 0 - as filtered clauses do not get scored

_search = [ { $search: { compound : { filter : [
{ range: { path : "hours" , lt: 10 } } , { text : { path: "activity", query: "ride" } }
] } } }, showscore ]

db.example.aggregate(search).pretty()
```

The **filter** operator works in the same ways as the **must** operator—a logical AND where all conditions must be true.

The difference between **filter** and **must** is that filter does not contribute to the score. This can be important when performing basic boolean filtering.

Using the ONLY filter will result in a score of 0 but results still being returned.



Compound can be combined

We can have multiple blocks for things we must have and things that are optional

```
showscore = { $set: { score: { $meta: "searchScore" } } }

search = [ { $search: { compound : {
  must : [ { text : { path: "activity", query: "watch" } }],
  should: [ { text : { path: "activity", query: "television" } }],
  filter: [{ range: { path : "hours" , gt: 10 } } ]
}}}, showscore]

db.example.aggregate(search).pretty()
```

50

compound clauses can be nested in the same way boolean logic can be nested to say "A" and "B" OR "C" and "D."

Anywhere you can put a search operator, range/test/wildcard/geoShape or similar, you can put a compound clause.

You can also apply different weights to different parts of the query using **score** to emphasize portions in the overall relevance score.



Compound can be nested

```
showscore = { $set: { score: { $meta: "searchScore" } } }

ten_hours_youtube = {compound:{ must : [ { text: { path: "activity", query: "YouTube"
}}, { range: { path:"hours" , gt: 10 }} ] }}

eight_hours_tv = {compound: { must : [ { text: { path: "activity", query: "television"
}}, { range: { path:"hours" , gt: 8 }} ] }}

search = [ { $search: { compound : {
  must : [ { text: { path: "activity", query: "watch"}}],
  should : [ ten_hours_youtube,eight_hours_tv], minimumShouldMatch: 1 } }}, showscore]

db.example.aggregate(search).pretty()
```

51

compound clauses can be nested in the same way boolean logic can be nested to say "A" and "B" OR "C" and "D."

Anywhere you can put a search operator, range/test/wildcard/geoShape or similar, you can put a compound clause.

You can also apply different weights to different parts of the query using **score** to emphasize portions in the overall relevance score.



Autocomplete, Highlighting, and Synonyms

52

When we search a Lucene index in Atlas Search, normally the `_id` field and relevance score for the document are returned to aggregation, and we get the documents that matched our search.

There are two other things we can do with Atlas Search autocomplete of queries and term highlighting.



What is Autocompletion?



53

Autocompletion is used in interactive user interfaces to help the user with suggestions about what they are searching for.

As we type into a box, a list of options are offered to save us typing in the full query—a user experience we are all familiar with.

To do this, we need to create a specially adapted Lucene index designed for autocompletion tasks. This is one of two types of **gram** indexes.

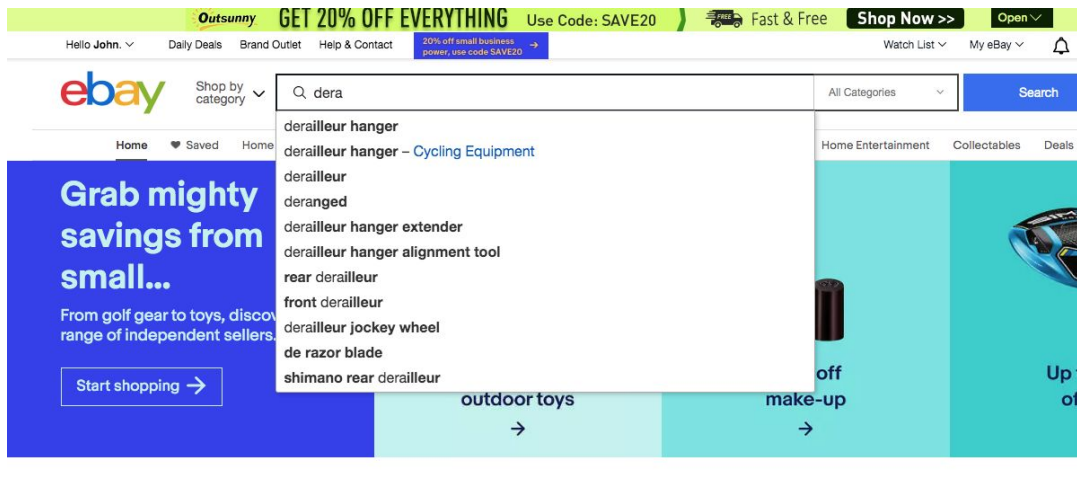
In a **gram** index, the index contains subsets of the field value not tokenized on words but rather on consecutive letters in those words. For example, with an nGram value 3 (also known as a trigram), if the field contains the word elephant, the index would have entries for ele, lep, eph, pha, han, ant.

Imagine an elephant reduced to the size of an ant.

We can then use this index to find field values where there are a minimum number of these tokens and also require them to be ordered—or not.

We can even apply fuzzy matching rules just like in our text search to accept small variations in the tokens.

Who needs Autocompletion?



54

Coming back to the nature of search versus database use cases, often, search users are trying to find information in the simplest way possible and may not have a good idea of what they are looking for, know the name of it, or be able to spell it.

Autocompletion is useful in any user interface where the user is entering free text strings to search for, as it can give faster and more immediate feedback to help them zero in on what they want.

In the most sophisticated cases of this, like internet search engines, many other factors are taken into account: what the user normally looks at, what they have recently looked at, what other things they have been doing, even things that nearby digital assistants have heard. With Lucene and Atlas Search, we are limited to offering things where the letters match, but it is still a powerful user interface tool.



Enabling autocomplete

We create the index like any other explicit index but use the data type **autocomplete**

This only works where the field contains a string

We select the shortest character sequence we want to have indexed

```
//Create a new index called  
'activities' using the JSON editor
```

```
{  
  "mappings": {  
    "dynamic": false,  
    "fields": {  
      "activity": {  
        "minGrams": 3,  
        "tokenization": "nGram",  
        "type": "autocomplete"  
      }  
    }  
  }  
}
```

55

To enable autocompletion, we need to create a special kind of index called an autocomplete index. For this, we can optionally specify a few parameters.

minGrams specifies the shortest character sequence we create an entry for. The recommendation here is at least 4 for a standard nGram to avoid the index being too large, although we will use 3 in our example.

We can specify maxgram as the longest sequence of characters to index.

We can also specify the **tokenization** method as edgeGram, rightEdgeGram or nGram.

- With an edgeGram, we tokenize at the start of each word, delimited by whitespace.
- With an rightEdgeGram, we tokenize at the end of each word, delimited by whitespace.
- With a nGram, we have a sliding window over the text and include spaces. We will look into this more in a moment.

Once again, we will create this **using the JSON editor** to simplify the process. **Make sure you call this index 'activities,' not 'default.'** All these options can be accessed via the GUI, specifically by turning off automatic mapping and selecting the datatype for the people field as autocomplete.

Note: We don't need necessarily a new index for this. We could just add autocomplete to the existing default index.



Using autocomplete

We can then type any 3-letter or longer sequence and find matching activities

We can also add some fuzziness so "liv" finds Climbing and television when we allow 1 change

```
search = [ { $search: {index: "activities", autocomplete: {
  path: "activity",
  query:"wat" }} }, { $project:{activity:1,_id:0}}]
```

```
db.example.aggregate(search).pretty()
```

```
search = [ { $search: {index: "activities", autocomplete: {
  path: "activity",
  query:"ing" }} }, { $project:{activity:1,_id:0}}]
```

```
db.example.aggregate(search).pretty()
```

```
search = [ { $search: {index: "activities", autocomplete: {
  path: "activity",
  query:"liv" }} }, { $project:{activity:1,_id:0}}]
```

```
db.example.aggregate(search).pretty()
```

```
search = [ { $search: {index: "activities", autocomplete: {
  path: "activity",
  query:"liv" , fuzzy: { maxEdits:1}}} }, { $project:{activity:
1,_id:0}}]
```

```
db.example.aggregate(search).pretty()
```

56

When we want to retrieve a list of autocomplete values. We need to explicitly state this when calling search. We do this using the autocomplete operator. Similar to the previous search operators, we specify it at the top level.

Normally, when using this, we will specify just a single field value to be returned using a projection.

The autocomplete operator knows how to use a specific ngram index and will return an error if one does not exist.

It is possible to use the fuzzy option to make partial matches as we can in a text search. Here, we search for 'liv' with fuzzy:{maxEdits:1}} and find Climbing and Television.

Autocomplete scores on the basis of how many edgGrams/rightEdgeGram/nGrams match in the field and can be combined with other fields.

edgeGrams, rightEdgeGrams and nGrams



When a string is broken down into matchable tokens for autocomplete, we use one of these methods

nGrams are all sets of consecutive letters and can include spaces

edgeGrams always start at the beginning of a word.

rightEdgeGrams always start at the end of a word.

String : "Quick fox"

nGrams (size 3-4)	edgeGrams (size 3-4)	rightEdgeGram (size 3-4)
Qui	Qui	fox
Quic	Quic	ick
uic	fox	uick
uick		
ick		
ick_		
ck_		
ck _f		

57

We use underscore (_) to mean space here in the example

- edgeGrams from the minimum to maximum size we specify will start at the beginning of each word.
- rightEdgeGrams from the minimum to maximum size we specify will start at the end of each word.
- nGrams make larger indexes but include all the inner letter pairs too.

The choice depends on whether you want to match just on the prefix of what people type or on any letter combination internally.



Four tips for using autocomplete

1. Consider the index size. It's easy to make one very large
2. nGrams create larger indexes than edgeGrams/rightEdgeGrams
3. The smaller the minimum gram size, the larger the index
4. To put exact matches at the top, combine with a phrase query

58

If your minSize is too small and especially if you are using **nGrams**, then you may end up with very large autocomplete indexes. Consider if you really need that level of autocompletion.

Using fuzzy will impact search performance as all the variations also need to be looked up in the index. By default, up to 50 variations of the term will be considered.

If you want exact matches to score highest, you should use **autocomplete** as a part of a **compound** query and combine it with a **phrase** with no **slop** - see <https://www.mongodb.com/docs/atlas/atlas-search/phrase/>



What is Highlighting?

Highlighting is way of showing the user why their search matched that document

It is particularly important where queries are against larger corpuses of text where the user may want to 'jump to match'

The screenshot shows the 'Search Tester' interface. At the top, there's a 'Search' button and a 'View Query Syntax' link. Below the search bar, three search results are displayed. Each result shows a score, a document ID, and a snippet of text. The first result has a score of 24.235015889140625 and ID '234556455'. The second result has a score of 20.41142082143555 and ID '22020940'. The third result has a score of 20.245137730161328 and ID '4589874'. Red arrows point to the highlighted text in each result: 'Bondi Beach' in the first, 'Bondi Beach!' in the second, and 'Bondi Beach!' in the third. The 'Matching fields' for each result are listed below the snippet.

Index Overview **Search Tester** [View Atlas Search Docs](#)

Enter a term below to run a simple \$search query on these sample documents from sample_airbnb_listings_and_reviews using your search index and see the top 10 documents that are returned ranked by score.

Search [RESET] [Search] [View Query Syntax]

SCORE: 24.235015889140625 _id: "234556455"

Enjoy an enviable lifestyle with Bondi Beach as your backyard.

Matching fields: space, reviews.comments, reviews.comments, reviews.comments, reviews.comments, description, description, summary, host.host_about, house.rules, description, access, interaction, summary, address.suburb, host.host_neighbourhood, address.street, transit, host.host_location, name, neighborhood_overview, notes, transit, host.host_about

SCORE: 20.41142082143555 _id: "22020940"

House Manual Welcome to unit 20/5 Campbell Parade, Bondi Beach!

Matching fields: house.rules, reviews.comments, reviews.comments, reviews.comments, reviews.comments, access, neighborhood_overview, host.host_about, address.suburb, host.host_neighbourhood, address.street, host.host_location, name, space, transit, description, summary, description, space, space, description

SCORE: 20.245137730161328 _id: "4589874"

Bondi Beach

Matching fields: amenities, reviews.comments, reviews.comments, reviews.comments, reviews.comments, reviews.comments, description, description, house.rules, summary, address.suburb, name, address.street, host.host_location, neighborhood_overview, summary, description, host.host_about, notes, host.host_about

Because search can match in a number of ways and can match against different parts of a document, sometimes it is useful to see the context of how a document matches.

By using the highlight option in conjunction with \$search, we can do this. Highlighted results are returned as part of the meta field the same way we return the score.



How to enable highlighting

Text is returned as an array
'texts' of objects with value and
type

Each array entry is either a 'hit'
where it matched the search or
'text' where it did not

```
MongoDB > search = [ { $search: {  
  text : { path: "activity", query: "watch" },  
  highlight : { path : "activity" } } },  
  { $set: { "highlights": {  
    "$meta": "searchHighlights" }  
  } }  
}]  
  
MongoDB > db.example.aggregate(search).pretty()  
  
highlights: [  
  {  
    score: 1.382846713066101,  
    path: 'activity',  
    texts: [  
      { value: 'watching', type: 'hit' },  
      { value: ' YouTube', type: 'text' }  
    ]  
  }  
]
```

60

To view the highlighting, we need to project the meta highlighting information the same way you projected the score.

We also need, as part of the search, to specify which fields we want highlighting in using highlight: {path:"X"}.

This returns an array with the relevant field being broken into the parts that matched and those that did not as an ordered array called **texts**.



Four tips for using highlighting

1. Highlighting shows what was matched, not what you searched for
2. Highlighting adds latency (30ms+) to a result and uses more CPU
3. If you do not need highlighting, you can make indexes smaller by removing the `store` option
4. To highlight the autocomplete indexed version of a path, the autocomplete operator must be the only operator that uses that path in the query

61

Your query will go through the usual processing, stemming, tokenizing, etc. so what matches may not be exactly what you typed.

This is expected but highlighting lets you see what did actually match or why a document was returned.

For Lucene to highlight, it needs to have maintained a copy of the indexed data—not just the inverted file indexing, but also a copy of the data. This is the default behaviour when making an index. This applies to a string index and therefore, when defining an index with a type string, you must not set 'store' to false, which reduces the size but prevents highlighting.

If you want smaller indexes and don't need highlighting, you can possibly cut your index size in half.

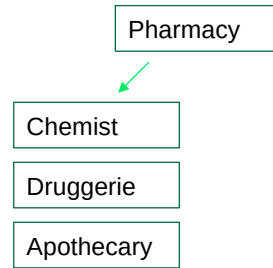


Synonyms

Allow alternative words to find data that means the same thing

Powerful context sensitive search capability

Atlas implementation makes it easy to define synonyms



62

Synonyms allow us to index and search our collection for words that have the same or nearly the same meaning.

To configure an Atlas Search index with synonym mappings, we:

- Add a new collection to hold the synonym definitions.
- Ensure this collection is in the same database as the index that will reference the synonym collection.
- Format the documents in the collection as described in the Synonyms Source Collection documentation.
- Reference the synonym source collection in a synonym mapping in the Search index definition.

This is actually much simpler than it sounds when we describe it.



How to define synonyms

First, we create a collection in the same database with synonyms we want to use

These can be **equivalent** terms or **explicit** one-directional mappings

```
MongoDB > example1 = { mappingType:"equivalent",synonyms: ["bike","bicycle","cycle"] }  
MongoDB > example2 = { mappingType:"explicit",input:["trek","giant","cube"], synonyms:["bike"] }  
MongoDB > db.activity_synonyms.drop()  
  
MongoDB > db.activity_synonyms.insertMany([example1,example2])
```

63

First, we create some documents in a collection that we are using to define what words have the same meaning as others.

This collection can be called anything but must be in the same database as the collection or collections we will use it with.

The documents have a specific format with either 2 or 3 required fields: `mappingType`, `synonyms`, and `input`.

For an **explicit** mapping, the word searched must be in **input**. Then, it will look for everything in **synonyms**.

For a **equivalent** mapping, the word searched must be in **synonyms**, and it will look for the other things in **synonyms**.

Atlas Search will automatically detect changes to this collection, but any malformed document (missing these fields) will cause a failure.

We can add additional metadata fields if we wish.



How to enable synonyms

Now we attach the synonym collection to a search index

Use the JSON editor to edit the default index and change it to use dynamic mapping and include our synonyms collection

We will also specify these synonyms are English words to allow it to appropriately stem them

```
//Replace default index
{
  "analyzer": "lucene.english",
  "searchAnalyzer": "lucene.english",
  "mappings": {
    "dynamic": true
  },
  "synonyms": [
    {
      "analyzer": "lucene.english",
      "name": "example",
      "source": {
        "collection": "activity_synonyms"
      }
    }
  ]
}
```

If we want to be able to use synonyms with an index, we must link the synonyms collection to it on index creation. We do this as in the example shown. Most of the time, you should specify the same analyzer for the synonyms as the index. Mixing analyzers can make it very hard to predict results.



Searching with synonyms

We must add **synonyms** to our text operator

And now we can search for "cycling" and will find "riding bikes" as:

cycling→cycle→bike→bikes

We can also look for "trek" or "cube," bicycle brand names

```
MongoDB > search = [ { $search: { text : { path: "activity", query: "cycling", synonyms: "example" }, highlight : { path : "activity" } } }, { $set: { "highlights": { "$meta": "searchHighlights" } } } ]

MongoDB > db.example.aggregate(search).pretty()

MongoDB > search = [ { $search: { text : { path: "activity", query: "trek", synonyms: "example" }, highlight : { path : "activity" } } }, { $set: { "highlights": { "$meta": "searchHighlights" } } } ]

MongoDB > db.example.aggregate(search).pretty()
```

65

When we query, our queries are analyzed (stemmed, etc.) and compared to the analyzed synonym list.

Then, the synonyms (which are returned pre-analyzed) are compared to the analyzed words in the document.

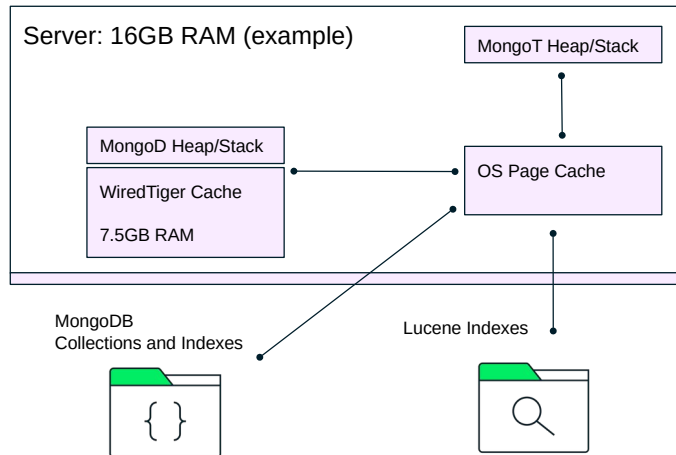
This is why "cycling" can find "bikes" although the synonym is "cycle" = "bike."

It is important to note that synonyms do not work with **fuzzy**. If using **text**, we must use one or the other.

Performance and Sizing



Factors that impact search performance



67

The biggest factor that will impact search performance is likely to be whether or not the relevant portions of the search indexes are in RAM at the time or need to be pulled from disk.

Atlas Search (mongot) is co-hosted with MongoDB (mongod). MongoDB will use ~50% of the RAM for its database cache and some of the remaining 50% to handle sorting, connections, aggregation, and other database tasks.

That additional 50% is also used to "cache" the Lucene indexes. These are in files accessed using virtual memory mapping techniques and therefore can be paged in and out as required. If RAM is needed for something else, then the cached portions will be evicted from memory. (They still exist on disk.) This page cache is shared with all OS processes pulling data from disk. It's basically a disk cache, including WiredTiger/MongoDB.

You can optimise this by ensuring you index only what you need, keeping the index sizes as viewed in the GUI well below 50% of the instance RAM size (20-25% is a reasonable amount) or increasing instance sizes as needed to fit your search indexes.



Index availability and consistency

Indexing is asynchronous

Indexing could fall a long way behind if the write load is high

An index will always be available even when re-indexing

68

Indexing is an asynchronous process watching changes that happen to the database using the Change Stream feature in MongoDB. If we have one or more indexes on a collection with a very high quantity of writes, then indexing may fall behind. If it falls off the oplog, then the collection will need to be reindexed completely.

Atlas will maintain a working index when re-indexing on a different node, so indexing is always available. This re-indexing happens if we run off the end of the oplog or change the index.

It is important to understand that indexing, unlike wiredTiger, is asynchronous and so we may not find the latest changes to our data immediately. This is not really an issue for typical "Search" use cases. Search is an index, not a database.



Important performance considerations

- Using a `$search` and then a `$match` or `$sort` in the pipeline is much slower
- Combine the `$match` into the `$search` using filter
- Use the `near` with `boost` operator to score in a way the same as your `$sort`

In-memory sort will use a lot of RAM and reduce Lucene index speed

Avoid using dynamic mapping on large documents with many keys

Results are sorted by score, so larger result sets need more resources

On a sharded cluster, all `$search` queries are sent to all shards

If not using highlighting, turn off the store option

69

If we need to find documents with search but then either sort or further filter them inside MongoDB, the performance will suffer. This will be an unindexed `$match` on each document and an in-memory `$sort`. We can push much of this work into `$search` to improve performance.

"Mapping explosions" occur when Atlas Search indexes a document with arbitrary keys and we have a dynamic mapping. The mongot process might consume increasing amounts of memory and could crash. If you add too many fields to an index, mapping explosions can occur. To address this issue, you can upgrade your cluster or use a static mapping that does not index all fields in your data.

The complexity level of Atlas Search queries and the type of operators used can affect database performance. Highly complex queries with multiple clauses are resource-intensive, as are queries which use the `regex` (regular expression) operator.

Atlas Search queries are ranked by score. Queries that return a large number of results are more computationally intensive because they must keep track of all the scores for the result set.



Reading explain plans

Explain plans show how a search runs and how scores are calculated

Add 'executionStats' argument to see timing information

Explain is divided into Context, Match, and Score

```
MongoDB > search={ $search : {  
  text : { query : "watching television", path: "activity"  
  }}}  
  
MongoDB > db.example.aggregate(search)  
  
MongoDB > db.example.explain().aggregate(search)  
  
MongoDB > db.example.explain('executionStats').aggregate(  
  search)
```

70

We look at the output of this in the next couple of slides.

The explain("queryPlanner") command, or even just explain(), returns the Lucene query tree that a \$search operator evaluates to, without timing information.

The explain("executionStats") and explain("allPlansExecution") commands return the time elapsed while performing each step of a search query, showing a breakdown of the time that each subcomponent took to execute.

The explain("executionStats") and explain("allPlansExecution") plans on Atlas Search queries expose information of three separate query execution areas:

Context: This shows time taken for tasks that create context used in execution of a Lucene query.

Match: The time spent in this area is related to the structure of the query, and is independent of the number of results that are iterated through and scored.

Score: This shows time taken for tasks related to iterating over and matching result documents; it shows the time it takes to determine which document is the next match.

Time spent matching results can vary significantly depending on the nature of the query. This includes time taken for tasks related to scoring documents.

Without ExecutionStats

We can see how the query is executed

We can see what comes out of the Analyzer

We can see the boolean logic
Should/Must etc



```
"stages" : [
  {
    "$_internalSearchMongotRemote" : {
      "mongotQuery" : {
        "text" : {
          "query" : "watching
television", "path" : "activity"
        }
      },
      "explain" : {
        "type" : "BooleanQuery",
        "args" : {
          "must" : [ ],
          "mustNot" : [ ],
          "should" : [
            {
              "type" : "TermQuery",
              "args" : {
                "path" : "activity",
                "value" : "watch"
              }
            },
            {
              "type" : "TermQuery",
              "args" : {
                "path" : "activity",
                "value" : "televiis"
              }
            }
          ]
        },
        "filter" : [ ],
        "minimumShouldMatch" : 0
      }
    }
  },
  {
    "$_internalSearchIdLookup" : {
    },
  },
],
```



With ExecutionStats

We see the context, match time, and score times

```
... TRIMMED ...
{
  "type" : "TermQuery",
  "args" : {
    "path" : "activity",
    "value" : "televis"
  },
  "stats" : {
    "context" : {
      "nanosElapsed" :
        NumberLong(118265),
      "invocationCounts" : {
        "createWeight" : NumberLong(1),
        "createScorer" : NumberLong(3)
      }
    },
    "match" : {
      "nanosElapsed" : NumberLong(0)
    },
    "score" : {
      "nanosElapsed" : NumberLong(1124),
      "invocationCounts" : {
        "score" : NumberLong(1)
      }
    }
  }
}
```




Upgrading your Atlas Cluster

Shared tier Atlas clusters are limited to three search indexes

Having a dedicated (M10 or higher) cluster will improve your search performance

For read-heavy search, favor RAM and CPU over Disk IOPS

For write-heavy use cases, Disk IOPS can be important too

Quiz Time!





#1. In which of the following scenarios would you use Atlas Search vs MongoDB query?

A

In a database of song lyrics, find a song you heard on the radio

B

In a database of bank statements, find the name of a restaurant you went to

C

In a database of hotels, find one with a pool and archery as activities

D

In a database of customers, find the highest spenders and what they buy

E

Display pop-up adverts on a website

Answer in the next slide.



#1. In which of the following scenarios would you use Atlas Search vs MongoDB query?

A

In a database of song lyrics, find a song you heard on the radio

B

In a database of bank statements, find the name of a restaurant you went to

C

In a database of hotels, find one with a pool and archery as activities

D

In a database of customers, find the highest spenders and what they buy

E

Display pop-up adverts on a website

If you need to run a query where you know exactly what you're looking for with an additional sort in mind, then using a query is the way to go as option D suggests. All other options can be handed by Atlas Search, including finding names of restaurant (implies a like) and even curating ads based on previous searches.



#2. You have a Knowledge base with MDB Atlas with thousands of articles. What are some advantages of using Atlas Search?

A

It reduces the storage cost of your database

B

It simplifies database schema design

C

It allows for fine-grained access control

D

It provides relevant ranking and scoring of search results

E

It provides search suggestions and allows synonyms in your queries

Answer in the next slide.

#2. You have a Knowledge base with MDB Atlas with thousands of articles. What are some advantages of using Atlas Search?

A

It reduces the storage cost of your database

B

It simplifies database schema design

C

It allows for fine-grained access control

D

It provides relevant ranking and scoring of search results

E

It provides search suggestions and allows synonyms in your queries

Atlas Search has a cost in terms of storage and doesn't make schema design any simpler. Additionally, there isn't the notion of access control like we typically have with other roles in the database. D & E are correct in providing results that are relevant and sorted based on the scoring and allows features like synonyms within the query to be handled.



#3. Select two true statement about the autocomplete feature in Atlas Search

A

Autocomplete is automatically set for all searches

B

The index must include a minGrams parameter (set to 4 or more)

C

It is only applicable to autocomplete indexes

D

It only works if the field has a string value

E

Tokenization method must be specified to edgeGram or nGram in the search

Answer in the next slide.



#3. Select two true statement about the autocomplete feature in Atlas Search

A

Autocomplete is automatically set for all searches

B

The index must include a minGrams parameter (set to 4 or more)

C

It is only applicable to autocomplete indexes

D

It only works if the field has a string value

E

Tokenization method must be specified to edgeGram or nGram in the search

Autocomplete indexes are a type of index that can be used in Atlas Search and has to be defined before calling it within the search stage via the aggregation. Additionally, the field that is to be indexed can only be of string type, numbers, boolean, object, etc. cannot be indexed with this type of index. MinGram is recommended to be set at 4 or more, but can be lower if you wish and there is also a rightEdgeGram option making E an incorrect choice.

#4. Which of these Atlas searches return this document using lucene.english analyzer and assuming there is a search index for all these fields?
{class:"Mathematics", hours: 280, type:"Numbers and Science"}



A

```
{ $search: { compound:
  { should: [
    { range: { path:
      "hours", lt: 10 } },
    { text : { path:
      "type", query:
      "science" }}} ] }
```

B

```
{ $search: { compound:
  { must: [
    { range: { path:
      "hours", gt: 10 } },
    { text : { path:
      "type", query:
      "science" }}} ] }
```

C

```
{ $search: { compound:
  { mustNot: [
    { range: { path:
      "hours", lt: 10 } },
    { text : { path:
      "class", query:
      "Maths" }}} ] }
```

D

```
{ $search: { text:
  { query: "Math",
    path: "class" ,
    fuzzy :
    { maxEdits:1 } } }
```

E

```
{ $search: { text:
  { query: "Matematic",
    path: "type" ,
    fuzzy :
    { maxEdits:1 } } }
```

Answer in the next slide.

#4. Which of these Atlas searches return this document using lucene.english analyzer and assuming there is a search index for all these fields?
`{class:"Mathematics", hours: 280, type:"Numbers and Science"}`

A

```
{ $search: { compound:
{ should: [
{ range: { path:
"hours", lt: 10 } },
{ text : { path:
"type", query:
"science" }}} ] }
```

B

```
{ $search: { compound:
{ must: [
{ range: { path:
"hours", gt: 10 } },
{ text : { path:
"type", query:
"science" }}} ] }
```

C

```
{ $search: { compound:
{ mustNot: [
{ range: { path:
"hours", lt: 10 } },
{ text : { path:
"class", query:
"Maths" }}} ] }
```

D

```
{ $search: { text:
{ query: "Math",
path: "class" ,
fuzzy :
{ maxEdits:1} } } }
```

E

```
{ $search: { text:
{ query: "Matematic",
path: "class" ,
fuzzy :
{ maxEdits:1} } } }
```

Remember the english analyzer tokenizes the words within the string differently so shortening the query to just 'math' will return the document. The compound operator also allows you to query with additional operators such as range, so retrieving values for hours greater than 10 will yield result. If the compound operator is not looking for specific things, the opposite can still be true, hence why C also returns a result.

For D and E there's a lot going on:

D will not return a result because "Mathematics" stems to "mathema", and there is no way to match "math" to "mathema". For this reason, we would need to define a synonym.

E works because the query gets stemmed to "matema", which is only 1 edit away from "mathema".

Remember that stemming is based on automatic rules to discard suffixes - see Analyzers and Tokens.

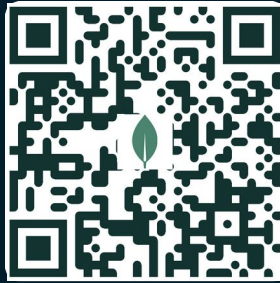
SKILL BADGE ASSESSMENT – 10 MINUTES



Search Fundamentals

MongoDB Skill

Search
Fundamentals



Take the Assessment

Questions



Ask your neighbor



Raise your hand



Let's do it together!

<https://mdb.link/Skill-SearchFundamentals-PS>

Recap

Atlas Search is an Atlas feature that creates indexes to support Search using Apache Lucene.

Search results are scored to rate how well they match the search.

Atlas search includes many search operators that can be combined. It also includes autocomplete, synonyms, and highlight capabilities.